



Chapter 6 — API & Real-Time Communication Design

6.1 API Design Principles

- **REST for commands and queries, WebSocket for live updates.** The UI never polls.
- **Contract-first:** every request/response is a Pydantic model; OpenAPI docs are auto-generated at `/docs`.
- **GeoJSON on the wire** for all spatial payloads — MapLibre consumes it natively.
- **Versioned:** all routes under `/api/v1/`; breaking changes require `/api/v2/`.
- **Consistent envelopes:** errors return `{ "error": { "code", "message", "details" } }` with correct HTTP status.

6.2 REST Endpoint Catalog

Auth & users

Method	Path	Description	Roles
POST	<code>/api/v1/auth/login</code>	Exchange credentials for access + refresh JWT	all
POST	<code>/api/v1/auth/refresh</code>	Rotate access token	all
GET	<code>/api/v1/users/me</code>	Current profile & role	all
POST	<code>/api/v1/users</code>	Create user	admin

Scenarios & runs (simulation control)

Method	Path	Description	Roles
GET	<code>/api/v1/scenarios</code>	List scenario definitions	all

Method	Path	Description	Roles
POST	<code>/api/v1/scenarios</code>	Upload/author scenario (YAML)	admin
POST	<code>/api/v1/runs</code>	Create run from scenario (with seed)	admin, planner
POST	<code>/api/v1/runs/{id}/start\</code>	pause\	resume\
PATCH	<code>/api/v1/runs/{id}/speed</code>	Set speed multiplier	admin, planner
GET	<code>/api/v1/runs/{id}/snapshot</code>	Full current state (units, incidents, weather, resources) — used on connect/reconnect	all
GET	<code>/api/v1/runs/{id}/replay?from=&to=</code>	Historical events for replay slider	all

Map & entities

Method	Path	Description
GET	<code>/api/v1/runs/{id}/units</code>	GeoJSON FeatureCollection of units
GET	<code>/api/v1/runs/{id}/units/{unitId}/track</code>	Movement history as LineString
GET	<code>/api/v1/runs/{id}/incidents?status=&severity=</code>	Incident list/GeoJSON
PATCH	<code>/api/v1/incidents/{id}</code>	Acknowledge / resolve (operator action → emits event)
GET	<code>/api/v1/runs/{id}/zones</code>	Sectors, restricted, hazard zones
POST	<code>/api/v1/runs/{id}/zones</code>	Draw new zone (GeoJSON polygon)
GET	<code>/api/v1/runs/{id}/events?after_id=&type=&limit=</code>	Event feed pagination + WS gap-fill

Missions & planning

Method	Path	Description
POST	<code>/api/v1/runs/{id}/missions</code>	Create draft mission (zone or waypoints)
POST	<code>/api/v1/missions/{id}/optimize</code>	Run A* optimization; returns route + estimates + comparison vs naive
POST	<code>/api/v1/missions/{id}/launch</code>	Hand mission to simulation engine
GET	<code>/api/v1/missions/{id}</code>	Mission detail with live progress

AI

Method	Path	Description
POST	<code>/api/v1/runs/{id}/ai/sitrep</code>	Generate situation report now
GET	<code>/api/v1/runs/{id}/ai/sitreps</code>	Report timeline
POST	<code>/api/v1/ai/chat</code>	Operator Q&A (SSE token stream); body includes run context flag
POST	<code>/api/v1/ai/rag/query</code>	Knowledge-base question → answer + source chunks
POST	<code>/api/v1/ai/rag/documents</code>	Ingest document (chunk + embed)

Vision, logistics, health, analytics

Method	Path	Description
POST	<code>/api/v1/vision/jobs</code>	Submit video for processing
GET	<code>/api/v1/vision/jobs/{id}</code>	Status + progress
GET	<code>/api/v1/vision/jobs/{id}/detections</code>	Detections (filter by class/risk)
PATCH	<code>/api/v1/detections/{id}</code>	Confirm / dismiss
GET	<code>/api/v1/runs/{id}/resources</code>	Inventory levels + burn rates
GET	<code>/api/v1/runs/{id}/personnel/vitals</code>	Latest vitals per person
GET	<code>/api/v1/runs/{id}/weather</code>	Active cells + 6h forecast
GET	<code>/api/v1/runs/{id}/forecasts?target=</code>	Analytics forecasts with intervals
GET	<code>/api/v1/runs/{id}/after-action</code>	Post-run performance report

6.3 WebSocket Protocol

Endpoint: `wss://host/ws?token=<JWT>` — one connection per client.

Client → server messages

```
{ "op": "subscribe", "run_id": "...", "topics": ["positions", "incidents", "alerts", "weather", "logistics", "ai"] }
{ "op": "unsubscribe", "topics": ["positions"] }
{ "op": "resume", "run_id": "...", "last_event_id": 48213 }
{ "op": "ping" }
```

Server → client messages

```
// High-frequency position frame (batched at 4 Hz)
{ "t": "positions", "run_id": "...", "sim_time": "...",
  "units": [ { "id": "u1", "lon": 77.59, "lat": 12.97, "heading": 45, "speed": 32 } ] }

// Discrete event (never dropped)
{ "t": "event", "id": 48214, "type": "IncidentCreated", "severity": "critical",
  "sim_time": "...", "geom": { "type": "Point", "coordinates": [77.6, 12.98] },
  "payload": { "kind": "medical", "title": "Evacuation requested" } }

// AI output notification
{ "t": "ai", "kind": "sitrep_ready", "sitrep_id": "..." }

// Control acknowledgements & errors
{ "t": "ack", "op": "subscribe", "ok": true }
{ "t": "error", "code": "UNAUTHORIZED_TOPIC", "message": "..." }
```

Reliability rules

1. Every discrete event carries the monotonically increasing DB `id`; clients track `last_event_id`.
2. On reconnect: client calls `GET /runs/{id}/snapshot`, then sends `resume` with `last_event_id`; the server replays the gap from the `events` table. **No missed**

incidents, ever.

3. Position frames are *lossy by design* (only the latest matters); events are *lossless*.
4. Heartbeat ping/pong every 20 s; stale connections closed at 60 s.

6.4 Canonical Event Schema

Every event in the system — DB row, bus message, WS push — shares this shape:

```
{
  "id": 48214,
  "run_id": "uuid",
  "type": "IncidentCreated",
  "source": "simulation | vision | operator | ai",
  "sim_time": "2026-07-21T14:32:00Z",
  "entity_id": "uuid | null",
  "geom": "GeoJSON | null",
  "severity": "info | warning | critical | null",
  "payload": { }
}
```

Event type registry (v1)

Domain	Types
Units	UnitSpawned , UnitMoved , UnitStatusChanged , UnitReportFiled
Incidents	IncidentCreated , IncidentAcknowledged , IncidentResolved
Missions	MissionPlanned , MissionLaunched , WaypointReached , MissionCompleted , MissionAborted
Weather	WeatherCellSpawned , WeatherCellMoved , WeatherCellDissipated
Logistics	ResourceConsumed , ResourceThresholdBreached , ResupplyCompleted
Health	VitalsSampled , HealthAlertRaised , HealthAlertCleared
Vision	DetectionCreated , DetectionConfirmed , DetectionDismissed
AI	SitrepGenerated , RecommendationIssued
System	RunStarted , RunPaused , RunResumed , RunCompleted

Adding a new event type requires: Pydantic model + registry entry + reducer case in the frontend. Nothing else — by design.

6.5 Bus Topics (Redis → Kafka mapping)

Redis channel (Phases 1–6)	Kafka topic (Phase 7)	Producers → Consumers
<code>evt.positions.{run}</code>	<code>doip.positions</code>	sim → gateway
<code>evt.incidents.{run}</code>	<code>doip.incidents</code>	sim, vision → gateway, ai, analytics
<code>evt.logistics.{run}</code>	<code>doip.logistics</code>	sim → gateway, analytics
<code>evt.health.{run}</code>	<code>doip.health</code>	sim → gateway, ai
<code>evt.weather.{run}</code>	<code>doip.weather</code>	sim → gateway, missions
<code>evt.ai.{run}</code>	<code>doip.ai</code>	ai → gateway
<code>jobs.vision</code>	<code>doip.vision.jobs</code>	api → vision workers

6.6 Rate Limiting & Pagination Standards

- Default rate limit: 100 req/min per user (Redis token bucket); AI endpoints 10 req/min; login 5/min per IP.
- List endpoints use cursor pagination: `?after_id=&limit=` (max 500); responses include `next_cursor`.
- Long-running operations (vision jobs, optimization > 2 s) return `202 Accepted` + job resource to poll or watch via WS.